A04｜下一次从哪里来，又在哪里停?

A03 处理一条记录的去向；A04 处理多轮工作。**循环负责继续，列表和字典组织内容，跨轮变量保留过程中的事实。** 它们目前在内存中，不会在程序结束时自动存入文件。

当前问题	关系或短用法	记到什么程度
下一轮怎样发生?	`while` 回到条件重查，但不会替代码更新条件依赖的值；`for item in items` 每轮取得下一项，没有下一项便结束。	按来源选择：检查是否还要继续，或依次处理来源中的项目。`for` 不只遍历列表。
怎样保留录入，不只剩最后一项?	`records = []` 建列表；`records.append(record)` 加到末尾。列表在收集前建立，不在每轮清空。	用熟添加。`append` **改变列表，返回 `None`；别把返回值再赋给列表名字。**
按位置取值，为什么容易越界?	`names[0]` 取第一项，`len(names)` 是数量。`range(3)` 提供 0、1、2，不含 3；`range(len(names))` 提供位置，不是姓名。	认得从 `0` 开始、上限不包含；只需要项目本身时直接遍历。
按编号或字段查，用什么组织?	字典按键取值，如 `status_by_id["R102"]`；同键再次赋值替换旧值。`key in mapping` 判断键是否存在。	分清位置与键；直接读取不存在的键会产生 `KeyError`，不是自动忽略。
多人、每人多字段，怎样连接?	列表保存多条字典；`records[0]["status"]` 先取第一条，再取状态字段，是两次取值。	沿结构找值；容器不自动保证字段完整或内容有效。
怎样记住“前面曾出现候补”?	标记在循环前初始化为 `False`，见到候补改为 `True`；后续不应无故清回去。	**状态放哪里、何时更新，要看它记一轮还是整个过程。** 不给每个循环强加标记。
提前停循环，与交回结果有什么不同?	`break` 离开最内层当前循环，仍可执行循环后代码；`return` 离开整个当前函数，交回调用处。	能指出下一步去哪；空来源也可能一次都不进入循环。

嵌套时，外层每次走到内层，都会开始这次内层遍历；内层 `break` 不自动结束外层。座位例见「每行还有几个座位，内层循环会重新开始吗？」。

录入程序保存了工作人员填写的状态，不代表已校验状态。结束词 `end` 要先判断再添加，避免当成姓名；录入与报告可分两段，由列表交接。

回看：推进见「同一段代码，为什么会再执行一次？」；添加见「append 与字符串清理的结果为什么不同？」；取值见「需要第几位时，位置和人数有什么区别？」「知道报名编号，不知道它排第几位，怎样查状态？」；状态见「怎样在循环结束以后，仍记得出现过候补？」。